

Université de Nouakchott
Master Intelligence Artificielle

CNN Avancés et Augmentation Moderne

De ResNet à ConvNeXt : Optimisation et Régularisation

Amanetoullah Cheikh(C22643) Oumar Med Vall(C22454) Hashimi(C21452)

Sous la direction de : Prof. Rifaa Sadegh

Plan de la Présentation

1. Motivation et Cas d'Usage Local
2. Le Problème de la Profondeur
3. L'Architecture ResNet
4. EfficientNet : L'Échelle Harmonieuse
5. ConvNeXt : La Modernisation des CNN
6. Stratégies de Régularisation et Augmentation
7. Algorithme et Pseudo-code
8. Implémentation Logicielle

Motivation et Cas d'Usage Local

Le Défi Contextuel :

- Diagnostic précoce des maladies du **palmier dattier** (ex : le Bayoud dans l'Adrar) ou suivi des cultures par drones le long du fleuve Sénégal.

Le Défi Contextuel :

- Diagnostic précoce des maladies du **palmier dattier** (ex : le Bayoud dans l'Adrar) ou suivi des cultures par drones le long du fleuve Sénégal.
- **Contrainte majeure** : Rareté des données labellisées (très petits datasets locaux) et conditions climatiques extrêmes (poussière, contrastes violents).

Le Défi Contextuel :

- Diagnostic précoce des maladies du **palmier dattier** (ex : le Bayoud dans l'Adrar) ou suivi des cultures par drones le long du fleuve Sénégal.
- **Contrainte majeure** : Rareté des données labellisées (très petits datasets locaux) et conditions climatiques extrêmes (poussière, contrastes violents).

Motivation : IA Vision au service de la Mauritanie

Le Défi Contextuel :

- Diagnostic précoce des maladies du **palmier dattier** (ex : le Bayoud dans l'Adrar) ou suivi des cultures par drones le long du fleuve Sénégal.
- **Contrainte majeure** : Rareté des données labellisées (très petits datasets locaux) et conditions climatiques extrêmes (poussière, contrastes violents).

Pourquoi les CNN classiques échouent ?

Les réseaux peu profonds (AlexNet, VGG) n'arrivent pas à capter les textures fines des maladies foliaires sous de fortes variations lumineuses. À l'inverse, les rendre simplement plus profonds détruit le signal.

Le Problème de la Profondeur

Le Paradoxe empirique (He et al., 2016) : Contre-intuitivement, ajouter des couches à un réseau de neurones classique entraîne une **augmentation de l'erreur d'entraînement**.

Le Paradoxe empirique (He et al., 2016) : Contre-intuitivement, ajouter des couches à un réseau de neurones classique entraîne une **augmentation de l'erreur d'entraînement**.

Les deux causes sous-jacentes

1. **Évanouissement du Gradient (Vanishing Gradient)** : Lors de la rétropropagation, les gradients sont multipliés par des matrices de poids. Si ces poids sont inférieurs à 1, le gradient tend exponentiellement vers 0.

Le Problème de la Profondeur : Dégradation & Gradient

Le Paradoxe empirique (He et al., 2016) : Contre-intuitivement, ajouter des couches à un réseau de neurones classique entraîne une **augmentation de l'erreur d'entraînement**.

Les deux causes sous-jacentes

1. **Évanouissement du Gradient (Vanishing Gradient)** : Lors de la rétropropagation, les gradients sont multipliés par des matrices de poids. Si ces poids sont inférieurs à 1, le gradient tend exponentiellement vers 0.
2. **Problème d'Optimisation** : Les espaces de paramètres profonds deviennent trop complexes à explorer pour la descente de gradient standard.

L'Architecture ResNet

ResNet : La Révolution de la Connexion Résiduelle

L'Idée Fondatrice : Pourquoi forcer les couches à apprendre une représentation brute alors qu'elles peuvent simplement apprendre un **ajustement** ?

ResNet : La Révolution de la Connexion Résiduelle

L'Idée Fondatrice : Pourquoi forcer les couches à apprendre une représentation brute alors qu'elles peuvent simplement apprendre un **ajustement** ?

Le Mécanisme du Skip Connection

On introduit une dérivation (raccourci) qui laisse passer le signal initial intact sans subir les transformations non linéaires de la couche.

ResNet : La Révolution de la Connexion Résiduelle

L'Idée Fondatrice : Pourquoi forcer les couches à apprendre une représentation brute alors qu'elles peuvent simplement apprendre un **ajustement** ?

Le Mécanisme du Skip Connection

On introduit une dérivation (raccourci) qui laisse passer le signal initial intact sans subir les transformations non linéaires de la couche.

Avantage théorique immédiat :

- Si une couche est inutile, le réseau peut simplement écraser ses poids à zéro. Le bloc se comporte alors comme une simple **fonction identité**, garantissant que les performances ne se dégraderont jamais avec la profondeur.

Formulation Mathématique du Bloc Résiduel

Soit x le vecteur d'entrée d'un bloc de couches. Le bloc cherche à modéliser la transformation idéale $H(x)$.

Formulation Mathématique du Bloc Résiduel

Soit x le vecteur d'entrée d'un bloc de couches. Le bloc cherche à modéliser la transformation idéale $H(x)$.

Équation Fondamentale

Plutôt que d'apprendre directement $H(x)$, on définit la fonction résiduelle $F(x) = H(x) - x$. Le bloc calcule alors :

$$y = F(x, \{W_i\}) + x$$

Formulation Mathématique du Bloc Résiduel

Soit x le vecteur d'entrée d'un bloc de couches. Le bloc cherche à modéliser la transformation idéale $H(x)$.

Équation Fondamentale

Plutôt que d'apprendre directement $H(x)$, on définit la fonction résiduelle $F(x) = H(x) - x$. Le bloc calcule alors :

$$y = F(x, \{W_i\}) + x$$

Pourquoi cela résout-il le problème du gradient ? À l'étape de rétropropagation, la dérivée par rapport à l'entrée devient :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \left(\frac{\partial F}{\partial x} + 1 \right)$$

Grâce au terme "+1", le gradient $\frac{\partial \mathcal{L}}{\partial y}$ se propage directement aux couches précédentes sans être altéré.

EfficientNet : L'Échelle Harmonieuse

EfficientNet : Le Principe du Compound Scaling

Le Problème traditionnel : Pour améliorer un CNN, on augmente soit sa profondeur (ResNet), soit sa largeur (Wide-ResNet), soit la résolution de l'image.

EfficientNet : Le Principe du Compound Scaling

Le Problème traditionnel : Pour améliorer un CNN, on augmente soit sa profondeur (ResNet), soit sa largeur (Wide-ResNet), soit la résolution de l'image.

L'approche de Tan & Le (2019) : L'Équilibre Mathématique
EfficientNet démontre que ces trois dimensions sont intimement liées. Si l'image est plus grande, le réseau a besoin de plus de canaux pour extraire les détails, et de plus de profondeur pour capter le contexte global.

EfficientNet : Le Principe du Compound Scaling

Le Problème traditionnel : Pour améliorer un CNN, on augmente soit sa profondeur (ResNet), soit sa largeur (Wide-ResNet), soit la résolution de l'image.

L'approche de Tan & Le (2019) : L'Équilibre Mathématique

EfficientNet démontre que ces trois dimensions sont intimement liées. Si l'image est plus grande, le réseau a besoin de plus de canaux pour extraire les détails, et de plus de profondeur pour capter le contexte global.

Profondeur : $d = \alpha^\phi$, Largeur : $w = \beta^\phi$, Résolution : $r = \gamma^\phi$

Sous la contrainte : $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ et $\alpha \geq 1, \beta \geq 1, \gamma \geq 1$

ConvNeXt : La Modernisation des CNN

ConvNeXt : Le Retour au Premier Plan des CNN

Le Contexte (2022) : Les Vision Transformers (ViT) commençaient à surpasser les CNN sur toutes les tâches de vision par ordinateur.

ConvNeXt : Le Retour au Premier Plan des CNN

Le Contexte (2022) : Les Vision Transformers (ViT) commençaient à surpasser les CNN sur toutes les tâches de vision par ordinateur.

La démarche de Liu et al. (2022)

Reprendre une architecture ResNet standard et lui appliquer, étape par étape, les choix de conception modernes des Transformers sans utiliser d'attention globale.

ConvNeXt : Le Retour au Premier Plan des CNN

Le Contexte (2022) : Les Vision Transformers (ViT) commençaient à surpasser les CNN sur toutes les tâches de vision par ordinateur.

La démarche de Liu et al. (2022)

Reprendre une architecture ResNet standard et lui appliquer, étape par étape, les choix de conception modernes des Transformers sans utiliser d'attention globale.

Les modifications clés intégrées :

- **Noyaux larges** : Passage de filtres 3×3 à des filtres de taille 7×7 (champs récepteurs élargis).

ConvNeXt : Le Retour au Premier Plan des CNN

Le Contexte (2022) : Les Vision Transformers (ViT) commençaient à surpasser les CNN sur toutes les tâches de vision par ordinateur.

La démarche de Liu et al. (2022)

Reprendre une architecture ResNet standard et lui appliquer, étape par étape, les choix de conception modernes des Transformers sans utiliser d'attention globale.

Les modifications clés intégrées :

- **Noyaux larges** : Passage de filtres 3×3 à des filtres de taille 7×7 (champs récepteurs élargis).
- **Inverted Bottleneck** : Canaux plus larges au milieu du bloc.

ConvNeXt : Le Retour au Premier Plan des CNN

Le Contexte (2022) : Les Vision Transformers (ViT) commençaient à surpasser les CNN sur toutes les tâches de vision par ordinateur.

La démarche de Liu et al. (2022)

Reprendre une architecture ResNet standard et lui appliquer, étape par étape, les choix de conception modernes des Transformers sans utiliser d'attention globale.

Les modifications clés intégrées :

- **Noyaux larges** : Passage de filtres 3×3 à des filtres de taille 7×7 (champs récepteurs élargis).
- **Inverted Bottleneck** : Canaux plus larges au milieu du bloc.
- **Simplification des activations** : Moins de fonctions d'activation (GELU au lieu de ReLU) et moins de Batch Normalization (remplacée par la Layer Normalization).

Stratégies de Régularisation et Augmentation

Augmentation de Données : Approche Standard

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Les transformations spatiales et photométriques

- **Flips horizontaux/verticaux** : Utile pour les plantes, interdit pour les chiffres.

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Les transformations spatiales et photométriques

- **Flips horizontaux/verticaux** : Utile pour les plantes, interdit pour les chiffres.
- **Random Resized Crop** : Simule des variations de distance de prise de vue.

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Les transformations spatiales et photométriques

- **Flips horizontaux/verticaux** : Utile pour les plantes, interdit pour les chiffres.
- **Random Resized Crop** : Simule des variations de distance de prise de vue.
- **Color Jitter** : Modifie la luminosité, le contraste et la saturation pour simuler les variations du soleil mauritanien.

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Les transformations spatiales et photométriques

- **Flips horizontaux/verticaux** : Utile pour les plantes, interdit pour les chiffres.
- **Random Resized Crop** : Simule des variations de distance de prise de vue.
- **Color Jitter** : Modifie la luminosité, le contraste et la saturation pour simuler les variations du soleil mauritanien.

Sur de petits volumes d'images (comme notre cas d'usage local), les réseaux mémorisent le bruit de fond au lieu des caractéristiques réelles.

Les transformations spatiales et photométriques

- **Flips horizontaux/verticaux** : Utile pour les plantes, interdit pour les chiffres.
- **Random Resized Crop** : Simule des variations de distance de prise de vue.
- **Color Jitter** : Modifie la luminosité, le contraste et la saturation pour simuler les variations du soleil mauritanien.

Limites : Ces méthodes ne modifient pas la nature intrinsèque de la frontière de décision, elles ne font que créer des duplications géométriques.

Principe (Zhang et al., 2018) : Forcer le réseau à se comporter de manière linéaire entre ses classes pour lisser la frontière de décision.

Principe (Zhang et al., 2018) : Forcer le réseau à se comporter de manière linéaire entre ses classes pour lisser la frontière de décision.

Formulation Mathématique

On sélectionne aléatoirement deux images (x_i, y_i) et (x_j, y_j) au sein du mini-batch, et on génère un nouvel échantillon virtuel $(\tilde{x}, \tilde{y})$:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j$$

$$\tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

Où le coefficient de mixage $\lambda \in [0, 1]$ suit une loi Bêta : $\lambda \sim \text{Beta}(\alpha, \alpha)$.

Principe (Zhang et al., 2018) : Forcer le réseau à se comporter de manière linéaire entre ses classes pour lisser la frontière de décision.

Formulation Mathématique

On sélectionne aléatoirement deux images (x_i, y_i) et (x_j, y_j) au sein du mini-batch, et on génère un nouvel échantillon virtuel $(\tilde{x}, \tilde{y})$:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

Où le coefficient de mixage $\lambda \in [0, 1]$ suit une loi Bêta : $\lambda \sim \text{Beta}(\alpha, \alpha)$.

Bénéfice : Le modèle n'apprend plus des prédictions absolues hyper-confiantes, ce qui réduit drastiquement le surapprentissage.

CutMix (Yun et al., 2019)

Au lieu de mélanger les pixels comme le Mixup (créant un effet fantôme), CutMix découpe une région de l'image A et la colle sur l'image B . Les étiquettes sont combinées proportionnellement à l'aire masquée.

- **Force** : Empêche le modèle de focaliser uniquement sur un détail distinctif majeur d'un objet.

CutMix (Yun et al., 2019)

Au lieu de mélanger les pixels comme le Mixup (créant un effet fantôme), CutMix découpe une région de l'image A et la colle sur l'image B . Les étiquettes sont combinées proportionnellement à l'aire masquée.

- **Force** : Empêche le modèle de focaliser uniquement sur un détail distinctif majeur d'un objet.

RandAugment (Cubuk et al., 2020)

Élimine la recherche manuelle fastidieuse de politiques d'augmentation.

- **Algorithme** : Définit un ensemble de N transformations. Pour chaque image, il choisit aléatoirement des transformations avec une magnitude globale M . Deux hyperparamètres seulement à régler.

Algorithme et Pseudo-code

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.
3. **Si** condition=CutMix **alors** appliquer le découpage spatial
 $\tilde{X} = \text{CutMix}(X_i, X_j)$

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
 2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.
 3. **Si** condition=CutMix **alors** appliquer le découpage spatial
 $\tilde{X} = \text{CutMix}(X_i, X_j)$
 4. **Sinon** appliquer le Mixup classique : $\tilde{X} = \lambda X_i + (1 - \lambda) X_j$
-

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
 2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.
 3. **Si** condition=CutMix **alors** appliquer le découpage spatial
 $\tilde{X} = \text{CutMix}(X_i, X_j)$
 4. **Sinon** appliquer le Mixup classique : $\tilde{X} = \lambda X_i + (1 - \lambda) X_j$
 5. Calculer la prédiction du réseau : $\hat{Y} = f_\theta(\tilde{X})$
-

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
 2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.
 3. **Si** condition=CutMix **alors** appliquer le découpage spatial
 $\tilde{X} = \text{CutMix}(X_i, X_j)$
 4. **Sinon** appliquer le Mixup classique : $\tilde{X} = \lambda X_i + (1 - \lambda) X_j$
 5. Calculer la prédiction du réseau : $\hat{Y} = f_\theta(\tilde{X})$
 6. Évaluer la perte interpolée : $\mathcal{L} = \lambda \text{Loss}(\hat{Y}, Y_i) + (1 - \lambda) \text{Loss}(\hat{Y}, Y_j)$
-

Pseudo-code : Pipeline d'Entraînement avec Augmentation

Initialisation : Modèle f_θ , Critère de perte (CrossEntropy), Optimiseur AdamW.

Pour chaque époque de l'entraînement :

1. Extraire un mini-batch d'images et labels originaux (X, Y)
 2. Échantillonner $\lambda \sim \text{Beta}(\alpha, \alpha)$ pour chaque paire d'images.
 3. **Si** condition=CutMix **alors** appliquer le découpage spatial
 $\tilde{X} = \text{CutMix}(X_i, X_j)$
 4. **Sinon** appliquer le Mixup classique : $\tilde{X} = \lambda X_i + (1 - \lambda) X_j$
 5. Calculer la prédiction du réseau : $\hat{Y} = f_\theta(\tilde{X})$
 6. Évaluer la perte interpolée : $\mathcal{L} = \lambda \text{Loss}(\hat{Y}, Y_i) + (1 - \lambda) \text{Loss}(\hat{Y}, Y_j)$
 7. Mettre à jour les poids par rétropropagation : $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
-

Implémentation Logicielle

Implémentation PyTorch : Le Bloc Résiduel From Scratch

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()
        self.conv_path = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
                      padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels)
        )
        # Ajustement des dimensions de l'identité si nécessaire
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        # Équation théorique :  $Y = F(x) + x$ 
        return nn.ReLU()(self.conv_path(x) + self.shortcut(x))
```

Utilisation en Production : Fine-Tuning via TIMM

```
import timm
import torch.nn as nn
from albumentations import Composite, RandomRotate90, Flip

# 1. Pipeline d'augmentation robuste avec Albumentations
transform = Composite([
    Flip(p=0.5),
    RandomRotate90(p=0.5)
])

# 2. Chargement de modèles avancés pré-entraînés
model_efficientnet = timm.create_model('efficientnet_b0', pretrained=True, num_classes=3)
model_convnext      = timm.create_model('convnext_tiny', pretrained=True, num_classes=3)

# 3. Accès direct aux tenseurs pour vérification du pipeline
print(f"Structure de sortie ConvNeXt : {model_convnext.head.fc}")
```

Étude Expérimentale et Résultats

L'ensemble des expérimentations a été mené sur un environnement cloud **Google Colab** équipé d'un **GPU NVIDIA T4** (Temps d'exécution ≤ 30 min).

L'ensemble des expérimentations a été mené sur un environnement cloud **Google Colab équipé d'un GPU NVIDIA T4** (Temps d'exécution ≤ 30 min).

Double validation sur Dataset

- **Dataset de référence** : CIFAR-10 (50 000 images standardisées en 10 classes).

L'ensemble des expérimentations a été mené sur un environnement cloud **Google Colab équipé d'un GPU NVIDIA T4** (Temps d'exécution ≤ 30 min).

Double validation sur Dataset

- **Dataset de référence** : CIFAR-10 (50 000 images standardisées en 10 classes).
- **Dataset Local** : Banque d'images collectées sur le terrain (palmiers de l'Adrar, cultures du Fleuve).

L'ensemble des expérimentations a été mené sur un environnement cloud **Google Colab équipé d'un GPU NVIDIA T4** (Temps d'exécution ≤ 30 min).

Double validation sur Dataset

- **Dataset de référence** : CIFAR-10 (50 000 images standardisées en 10 classes).
- **Dataset Local** : Banque d'images collectées sur le terrain (palmiers de l'Adrar, cultures du Fleuve).

L'ensemble des expérimentations a été mené sur un environnement cloud **Google Colab équipé d'un GPU NVIDIA T4** (Temps d'exécution ≤ 30 min).

Double validation sur Dataset

- **Dataset de référence** : CIFAR-10 (50 000 images standardisées en 10 classes).
- **Dataset Local** : Banque d'images collectées sur le terrain (palmiers de l'Adrar, cultures du Fleuve).

Stratégie d'Ablation systématique : Mesurer individuellement l'apport de l'augmentation moderne (avec/sans Mixup/CutMix) et l'importance critique des composants structurels (avec/sans normalisation par lots).

Résultats Comparatifs : Benchmark Global

Table 1: Performances relevées (Précision globale & Vitesse d'inférence)

Modèle	Augmentation	Précision CIFAR-10	Précision Dataset Local
ResNet-18	Standard	78.4%	71.2%
ResNet-18	Mixup + CutMix	83.1%	79.5%
EfficientNet-B0	RandAugment	84.5%	81.3%
ConvNeXt-Tiny	RandAugment	88.2%	84.6%

Résultats Comparatifs : Benchmark Global

Table 1: Performances relevées (Précision globale & Vitesse d'inférence)

Modèle	Augmentation	Précision CIFAR-10	Précision Dataset Local
ResNet-18	Standard	78.4%	71.2%
ResNet-18	Mixup + CutMix	83.1%	79.5%
EfficientNet-B0	RandAugment	84.5%	81.3%
ConvNeXt-Tiny	RandAugment	88.2%	84.6%

Observations majeures

- L'injection des augmentations modernes offre un gain net de +8.3% sur le petit jeu de données local.

Table 1: Performances relevées (Précision globale & Vitesse d'inférence)

Modèle	Augmentation	Précision CIFAR-10	Précision Dataset Local
ResNet-18	Standard	78.4%	71.2%
ResNet-18	Mixup + CutMix	83.1%	79.5%
EfficientNet-B0	RandAugment	84.5%	81.3%
ConvNeXt-Tiny	RandAugment	88.2%	84.6%

Observations majeures

- L'injection des augmentations modernes offre un gain net de +8.3% sur le petit jeu de données local.
- ConvNeXt-Tiny surclasse les autres modèles au prix d'un coût de calcul légèrement plus élevé.

Qu'arrive-t-il si l'on retire un composant clé ?

Qu'arrive-t-il si l'on retire un composant clé ?

1. L'impact de la BatchNorm (Ablation critique)

Le retrait total des couches 'BatchNorm2d' dans le bloc ResNet-18 provoque une **instabilité immédiate** du gradient à l'entraînement. La précision s'effondre à 41.0%.

Qu'arrive-t-il si l'on retire un composant clé ?

1. L'impact de la BatchNorm (Ablation critique)

Le retrait total des couches 'BatchNorm2d' dans le bloc ResNet-18 provoque une **instabilité immédiate** du gradient à l'entraînement. La précision s'effondre à 41.0%.

2. Synergie Mixup + CutMix

L'utilisation combinée empêche le réseau de sur-apprendre des motifs d'arrière-plan statiques (comme le sable ou le ciel du désert) et le force à se concentrer exclusivement sur la texture des feuilles malades.

Le Pont Conceptuel : Arbres vs Pixels

Comparaison Explicite : Connexion Résiduelle vs Boosting

Bien qu'appartenant à deux mondes mathématiques distincts, le Gradient Boosting (XGBoost) et le Deep Learning Résiduel (ResNet) partagent le même principe fondamental.

Caractéristique	Module ML : Boosting	Module DL : CNN Avancés
Concept central	Correction additive séquentielle.	Correction additive en profondeur.
Mécanisme	Chaque nouvel arbre $h_m(x)$ modélise le gradient négatif .	Chaque bloc $F(x)$ calcule un résidu correctif .
Formulation	$F_m(x) = F_{m-1}(x) + \rho h_m(x)$	$y = x + F(x)$

Comparaison Explicite : Connexion Résiduelle vs Boosting

Bien qu'appartenant à deux mondes mathématiques distincts, le Gradient Boosting (XGBoost) et le Deep Learning Résiduel (ResNet) partagent le même principe fondamental.

Caractéristique	Module ML : Boosting	Module DL : CNN Avancés
Concept central	Correction additive séquentielle.	Correction additive en profondeur.
Mécanisme	Chaque nouvel arbre $h_m(x)$ modélise le gradient négatif .	Chaque bloc $F(x)$ calcule un résidu correctif .
Formulation	$F_m(x) = F_{m-1}(x) + \rho h_m(x)$	$y = x + F(x)$

Synthèse pour le jury

Le boosting empile les modèles horizontalement dans le temps pour réduire le biais, tandis que ResNet empile les couches verticalement dans l'espace pour préserver la variance et le gradient.

Limites, Déploiement et Bonnes Pratiques

Les Pièges Fréquents :

- **Data Leakage via l'Augmentation** : Effectuer le Mixup ou le CutMix avant la séparation du jeu de test détruit la validité de l'évaluation. L'augmentation se fait uniquement en ligne sur le train-set.

Les Pièges Fréquents :

- **Data Leakage via l'Augmentation** : Effectuer le Mixup ou le CutMix avant la séparation du jeu de test détruit la validité de l'évaluation. L'augmentation se fait uniquement en ligne sur le train-set.

Stratégie de Déploiement Local (Edge Computing)

Les infrastructures réseau pouvant être instables dans les zones agricoles reculées, l'inférence doit s'effectuer **hors ligne (on-device)**.

Les Pièges Fréquents :

- **Data Leakage via l'Augmentation** : Effectuer le Mixup ou le CutMix avant la séparation du jeu de test détruit la validité de l'évaluation. L'augmentation se fait uniquement en ligne sur le train-set.

Stratégie de Déploiement Local (Edge Computing)

Les infrastructures réseau pouvant être instables dans les zones agricoles reculées, l'inférence doit s'effectuer **hors ligne (on-device)**.

Bonnes pratiques appliquées :

- **Quantization (INT8)** : Compression des poids FP32 en entiers 8 bits pour diviser la taille mémoire par 4 sans perte significative de précision.

Conclusion, Démo et Bibliographie

Conclusion et Références Bibliographiques i

Conclusion : Les CNN modernes armés d'augmentations régularisées constituent la solution la plus viable pour appliquer l'IA en vision sur des scénarios industriels ou agricoles souverains à faibles ressources.

Lectures Fondatrices :

- He, K. et al. (2016). *Deep Residual Learning for Image Recognition*. CVPR.
- Tan, M. & Le, Q. (2019). *EfficientNet : Rethinking Model Scaling*. ICML.
- Liu, Z. et al. (2022). *A ConvNet for the 2020s*. CVPR.
- Zhang, H. et al. (2018). *mixup : Beyond Empirical Risk Minimization*. ICLR.

Recherches Récentes (Orientations 2024-2025) :

- *YOLOv10 : Real-Time End-to-End Object Detection* (Wang et al., Mai 2024). Optimisation extrême de l'architecture pour l'inférence sur périphériques Edge sans NMS.
[Lien arXiv:2405.14458](#)
- *Mamba-Vision : A Pure State Space Model for Edge Devices* (Ali et al., Juillet 2024). Modèles d'états stables, alternative très performante et légère aux CNN et ViT classiques.
[Lien arXiv:2407.08083](#)
- *StarNet : Simple networks for vision* (Ma et al., Mars 2024). Simplification radicale des architectures CNN traditionnelles pour atteindre une efficacité maximale sur de petits jeux de données.
[Lien arXiv:2403.19967](#)